PARALELINUX 2

Primer avance de proyecto

Jaffet Araya Góngora C4C537
Jason Montenegro Navarro C4H386
Iván Ríos Baltonado C4I919



PROTOCOLO BTP

El protocolo BTP (Basic Transfer Protocol) es un protocolo
para envío de paquetes de información en red, con tres actores principales:

- **Clientes** → envían paquetes vía TCP a intermediarios, dirijidos a servidores
- **Intermediarios** → remiten paquetes de clientes a servidores, y viceversa
- **Servidores** → responden a paquetes de clientes por medio de intermediarios

PAQUETE BTP

El paquete BTP comprende los siguientes atributos:

- ip-type
- source-ip
- destination-ip
- packet-type
- content-type
- content-length
- content

Paquete BTP	Tamaños en bytes
[ip-type].....	1 byte
[source-ip].....	16 bytes
[destination-ip].....	16 bytes
[packet-type].....	1 byte
[content-type].....	1 byte
[content-length].....	4 bytes
[content].....	n bytes

VALORES DE ATRIBUTOS

content-type

- text/plain
- text/html/all-lego
- text/all-lego
- text/lego
- text/all-figures
- text/figure

packet-type

- STOP
- REQUEST
- RESPONSE
- UNREACHABLE

ip-type

- IPv4
- IPv6



PROTOCOLO DE BITÁCORAS

Ubicación: logs/

Nombre: archivo-#.log

Formato: [Fecha] – [Tipo de evento]: [Mensaje]

Tipos de evento

- INFO
- ERROR

Ejemplo

Mon Mar 23 07:05:57 2026 – [INFO]: Server redirecting packet to (192.168.0.10).

EZ-FILESYSTEM

Componentes

- **Super bloque** → metadatos globales del contenedor (total de bloques, tamaños, offsets)
- **Bitmap** → rastrea bloques: ocupados (1) o libres (0)
- **Inodos** → identificadores de archivos y directorios; apuntan a bloques de datos
- **Bloques de datos** → contenido real de los archivos

Tamaño del disco: [1024 bytes, ~1.099 TB]
Tamaño de bloque: 256 bytes
Tamaño por archivo:]0, ~1.0226 GB]
Archivo contenedor: ez_container.bin
Relación inodo/datos: 1 inode : 8 bloques de datos



ESTRUCTURA

Super block	Bitmap			Inode table	
file_system_identifier total_blocks block_size inode_table_start inode_count data_start blocks_available inodes_available padding[3]	[-][-][-]...[-]	...	[-][-][-]...[-]	Inode_type inode_id total_size created_timestamp direct_blocks[15] indirect_blocks[15] double_indirect_blocks[15] triple_indirect_blocks[15] padding[3]	...

Data blocks		
Raw data	...	Raw data

CASOS DE PRUEBA

Los casos de prueba se basan en el uso del protocolo BTP en el contexto de solicitudes de piezas Lego.



Los casos de prueba se subdividieron en tres categorías:

- Camino feliz
- Contenido malformado
- Red y conectividad

CAMINO FELIZ

ID	Caso	Cliente	Intermediario	Servidor	Resultado
1	Cliente envía un paquete `text/all-lego` con contenido vacío	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y responde con `text/all-lego` con todos los legos, sus cantidades y descripciones	El cliente recibe todos los legos almacenados en formato `"quantity":$,"description":"description text"` por cada entrada
2	Cliente envía un paquete `text/lego` con contenido igual a la descripción de un lego existente	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y responde con `text/lego` con la cantidad actual y descripción de esa pieza	El cliente recibe `"quantity":$,"description":"description text"` con los datos del lego solicitado
3	Cliente envía un paquete `text/all-figures` con contenido vacío	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y responde con `text/all-figures` con los nombres de todas las figuras que se pueden crear	El cliente recibe todas las figuras en formato `"name&#","name&#",...`

4	Cliente envía un paquete `text/figure` con contenido igual al nombre exacto de una figura existente	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y responde con `text/figure` con todas las piezas Lego que conforman esa figura	El cliente recibe las piezas en formato `"quantity":$,"description":"description text"` por cada pieza de la figura
5	Cliente envía un paquete `text/html/all-lego` con contenido vacío	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y responde con `text/html/all-lego` con todos los legos en formato HTML	El cliente recibe el inventario completo como contenido HTML válido
6	El inventario del servidor está vacío y el cliente solicita `text/all-lego`	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y responde con `text/all-lego` con `content-length: 0` y contenido vacío	El cliente recibe un paquete de respuesta válido pero sin ninguna entrada de legos
7	El inventario del servidor está vacío y el cliente solicita `text/all-figures`	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y responde con `text/all-figures` con los nombres de todas las figuras que se pueden crear	El cliente recibe todas las figuras en formato `"name&#","name&#",...`

CONTENIDO MALFORMADO

ID	Caso	Cliente	Intermediario	Servidor	Resultado
8	Cliente envía un paquete `text/lego` con contenido que no corresponde a ningún lego existente	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y no reconoce la pieza. Responde con `text/lego` indicando desconocido	El cliente recibe `"quantity":0,"description":"unknown lego"`
9	Cliente envía un paquete `text/lego` con contenido vacío	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete con descripción vacía. Al no reconocer ningún lego, responde con `text/lego` indicando desconocido	El cliente recibe `"quantity":0,"description":"unknown lego"`
10	Cliente envía un paquete `text/figure` con contenido vacío	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete con nombre de figura vacío. Al no reconocer ninguna figura, responde con `text/figure` indicando	El cliente recibe `"quantity":0,"description":"unknown figure"`

11	Cliente envía un paquete `text/figure` con contenido que no corresponde a ninguna figura existente	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y no reconoce la figura. Responde con `text/figure` indicando desconocida	El cliente recibe `"quantity":0,"description":"unknown figure"`
12	Cliente envía un paquete `text/lego` cuya pieza sí existe pero tiene cantidad 0 en el inventario	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete, reconoce la pieza pero no tiene unidades disponibles, responde con `text/lego`	El cliente recibe `"quantity":0,"description":"no inventory"`
13	Cliente envía un paquete `text/all-lego` con contenido no vacío	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y responde con `text/all-lego` con todos los legos, sus cantidades y descripciones	El cliente recibe todos los legos almacenados en formato `"quantity":$,"description":"description text"` por cada entrada
14	Cliente envía un paquete `text/all-figures` con contenido no vacío	Crea y envía el paquete al intermediario	Redirige el paquete del cliente al servidor, y la respuesta del servidor al cliente	Recibe el paquete y responde con `text/all-figures` con los nombres de todas las figuras que se pueden crear	El cliente recibe todas las figuras en formato `"name&#","name&#",...`

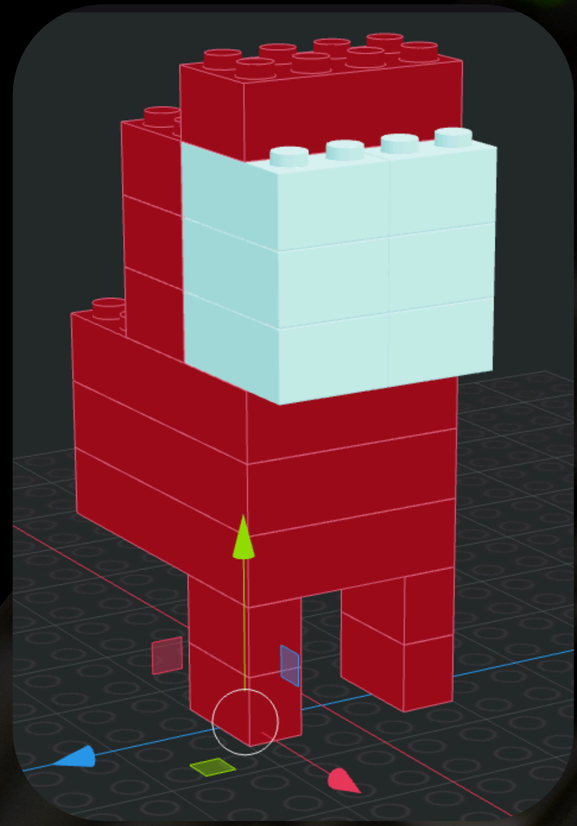
RED Y CONECTIVIDAD

ID	Caso	Cliente	Intermediario	Servidor	Resultado
15	Cliente envía un paquete cuya `destination-ip` no corresponde a ningún servidor conectado al intermediario	Crea y envía el paquete al intermediario	No encuentra ninguna conexión con la `destination-ip` indicada. Genera un paquete `UNREACHABLE` hacia el cliente	No participa	El cliente recibe un paquete de tipo `UNREACHABLE` con `content-type: text/plain` cuyo contenido es la IP del servidor inalcanzable
16	Cliente envía un paquete `STOP` al intermediario	Envía el paquete `STOP` y se desconecta	Recibe el paquete `STOP` y lo reenvía a todos sus nodos conectados para notificar la desconexión. No responde al cliente, y se desconecta	Recibe el paquete `STOP` y finaliza su ejecución	La red se desconecta ordenadamente; ningún actor intenta responder al cliente

PROPUESTA FIGURA PROPIA

Consiste en una figura de Among Us.

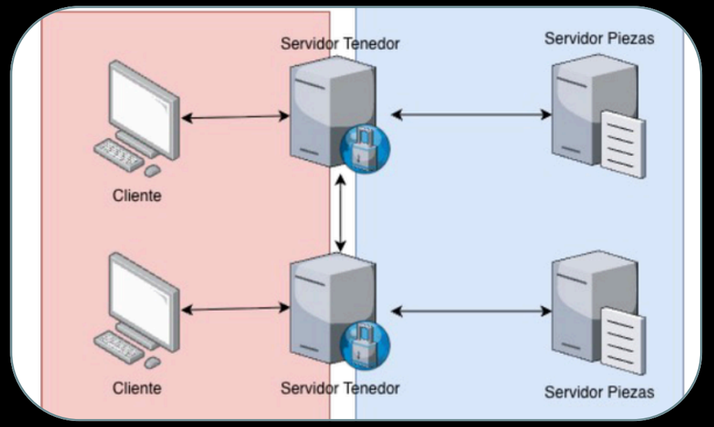
- **Parte 1:**
  - 4x brick 1x2 red
  - 3x brick 4x6 red
  - 4x brick 2x4 red
- **Parte 2:**
  - 6x brick 2x3 light blue
- Total 17 piezas.

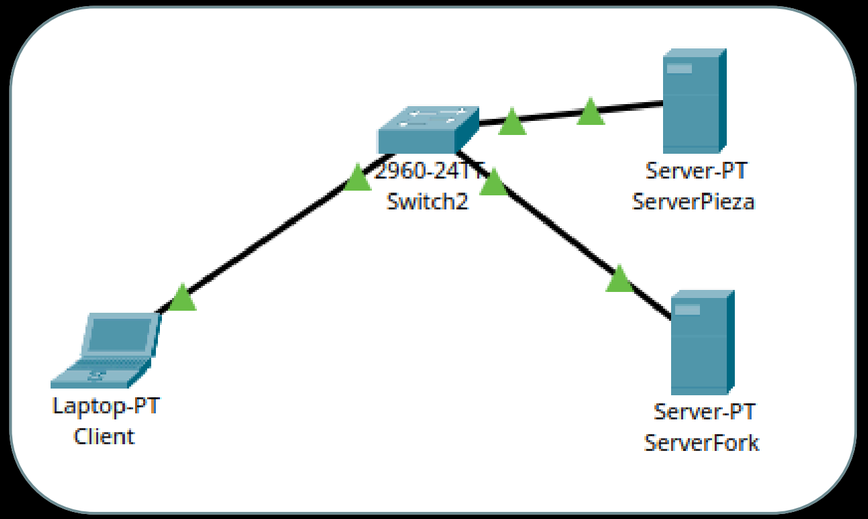


AMOGUS

DISEÑO DE PACKET TRACER

- Agentes principales
  - **Switch**: direccionamiento MAC, VLAN 220.
  - **Cliente**: solicitud de pieza
  - **Server Fork**: intermediario Cliente–Pieza
  - **Server Pieza**: atender solicitud.





CLIENTE LEGO



Un cliente es un programa que:

- **Solicita** servicios a un servidor
- **Envía** peticiones (requests)
- **Recibe** respuestas (responses)

Objetivo: conseguir piezas de lego para armar sus figuras.

- Usa protocolos como HTTP/ HTTPS
- Se comunica a través de sockets
- Cifrado SSL

PETICIÓN Y RESPUESTA

Petición del cliente

GET /lego/list.php?
figure=X&part=1 HTTP/1.1
Host: os.ecci.ucr.ac.cr
Connection: close

RESPUESTA DEL SERVIDOR

HTTP/1.1 200 OK
DATE: THU, 26 MAR 2026 04:37:53 GMT
SERVER: APACHE
X-POWERED-BY: PHP/5.4.16
CONTENT-LENGTH: 2193
CONNECTION: CLOSE
CONTENT-TYPE: TEXT/HTML; CHARSET=UTF-8
<HEAD>
<TITLE> FIGURAS DE LEGO </TITLE>
<HEAD>
<TITLE> CI-0123

SIMULACIÓN DE PROTOCOLO

```
Tipos de peticiones
        "PLAIN"          REQUEST text/plain
        "ALL_LEGO"       REQUEST text/all-lego
        "LEGO"           REQUEST text/lego
        "ALL_FIGURES"    REQUEST text/all-figures
        "FIGURE"         REQUEST text/figure
Para terminar la simulación, ingrese: "STOP".
Seleccione algún tipo de petición.
Cliente:FIGURE
Ingrese el contenido de su paquete REQUEST.
[FIGURE]: amogus&2

 Paquete enviado
=== Btp packet ===
ip-type: IPv4
source-ip: 127.0.0.1
destination-ip: 127.0.0.1
packet-type: REQUEST
content-type: text/figure
content-length: 8
content: amogus&2


 Paquete recibido
=== Btp packet ===
ip-type: IPv4
source-ip: 127.0.0.1
destination-ip: 127.0.0.1
packet-type: RESPONSE
content-type: text/figure
content-length: 49
content: "quantity":6,"description":"brick 2x3 light blue"

Tipos de peticiones
```

- Protocolo BTP.
- Utilizacion de colas thread-safe, manejadas por el intermediario.

Acciones:

- PLAIN: para pruebas.
- ALL_LEGO: inventario completo.
- LEGO: Buscar pieza especifica.
- ALL_FIGURES: todos los nombres de figuras disponibles.
- FIGURE: piezas requeridas para figura.
- STOP

GRACIAS